



Topic 1: MATLAB Basics

Jaber Taheri-Shakib

Research associate

Civil and Environmental Engineering

University of Waterloo | McMaster University

Variables

variable = expression

1	<code>year = input("What year is it?\n");</code>
---	--



Variables

variable name

A variable name in MATLAB:

1. Must start with a letter
2. Can contain letters, numbers, and underscores _
3. Cannot start with a number
4. Cannot Start With An Underscore
5. Cannot contain special characters such as -, @, %, or spaces
6. Should be meaningful and easy to understand



Variables

Do these names count?

#	MATLAB statement	Does it Work?	If not, why?	Is it a good name?
1	<code>asdf1 = 100;</code>	YES		unintelligible
2	<code>col-03 = 10;</code>	No	- is not acceptable	YES
3	<code>8val = 10;</code>	No	Cannot start with #	No
4	<code>aaaa3 = 10;</code>	YES		No
5	<code>_col = 0;</code>	No	Cannot start with _	Maybe
6	<code>row_3 = 1;</code>	YES		YES
7	<code>love = 10;</code>	YES		Maybe
8	<code>row@3 = 10;</code>	No	@ is not alphanumeric	Maybe

Variables

What's the output?

Input Script:

```
1 mass = 20;  
2 mass = mass + 5;  
3 fprintf('The object weighs %d kg.\n', Mass);
```

Output Command Window:

```
>> untitled  
Unrecognized function or variable 'Mass'.  
  
Error in untitled (line 3)  
fprintf('The object weighs %d kg.\n', Mass);
```

Part	Meaning
fprintf	Prints formatted text in the Command Window
(	Starts the command input
'The object weighs %d kg.\n'	The text format to be printed
The object weighs	Regular text
%d	Placeholder for an integer number
kg	Unit of measurement
\n	Moves to the next line
,	Separates the text format from the variable
Mass	The variable whose value replaces %d
)	Ends the command input
;	Suppresses extra output in MATLAB

Constants

- A **variable** stores a value that can be changed by the programmer.

Example:

`x = 5;`

`x = 10;`

- A constant is a predefined value with a special meaning in MATLAB.

Examples:

✓ `pi` (π) 3.14159...

✓ `i, j` imaginary numbers

✓ `inf` infinity

✓ `NaN` Not a Number

❑ Important:

Avoid using MATLAB predefined names as variable names.

Example:

`pi = 10;`

This is allowed, but it changes the meaning of `pi` in the current workspace.

To restore it:

`clear pi`

Overwriting Constants

- Constants are treated just like variables by MATLAB so you can overwrite values to the constants.
- The original constant can be restored by using the command `clear` to wipe the variable out of memory.

⚠: Overwriting constants is not recommended as it can cause problems later in your code.



Overwriting Constants

```
r = 2;
```

```
area1 = pi*r^2
```

```
area1 = 12.5664
```

```
clc  
clear
```

```
pi = 10;
```

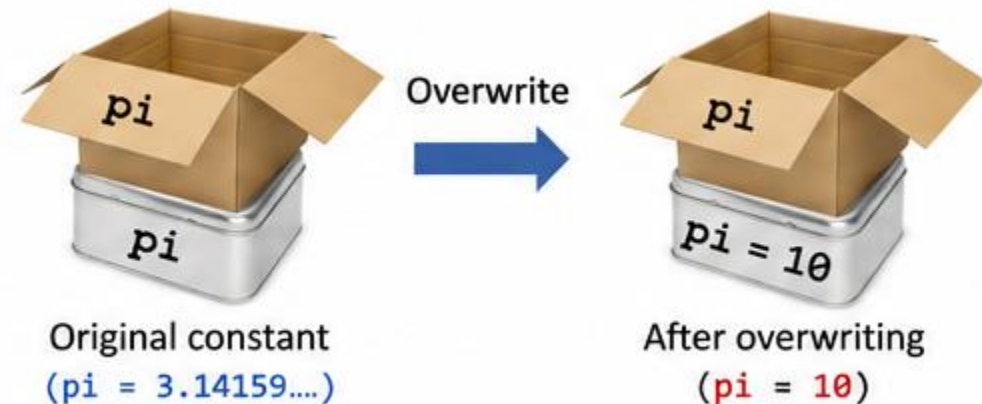
```
area2 = pi*r^2
```

```
area2 = 40
```

```
clear pi
```

```
area3 = pi*r^2
```

```
area3 = 12.5664
```



Data Types in MATLAB

A data type tells MATLAB what kind of value is stored.

1. Main idea

Every variable has a type, also called a class.

Think of the type as the shape and size of a box.

- shape: what the value can do
- size: how much memory it uses

2. Common types

double

decimal numbers
(default)

int

whole numbers

logical

true / false
(1 or 0)

char

single text
'c'

string

text words
"hello"

3. How to check a type

Use the MATLAB command:

```
class(variableName)
```

Example:

```
x = 3.14;  
class(x)
```

```
ans = double
```

Key message: In MATLAB, every value has a class. The class helps MATLAB know how to store and use that value.

Quick MATLAB Demo: Checking Data Types

Type these examples in the Command Window.

Please open MATLAB and enter the following code in the Command Window. We will run it together and check the results.

Number

```
x = 3.14;  
class(x)
```

Output:

```
ans = double
```

Whole number

```
n = int32(10);  
class(n)
```

Output:

```
ans = int32
```

Text

```
name = "MATLAB";  
class(name)
```

Output:

```
ans = string
```

True / False

```
flag = true;  
class(flag)
```

Output:

```
ans = logical
```

Tip: `class(x)` tells us the data type of `x`.

Creating Different Types of Variables

The value and the symbols you use tell MATLAB the variable type. Use `class()` to check it.

Simple idea

Every variable has a **type** or **class**.

The class tells MATLAB what kind of value is stored: number, text, or true/false.

Main rule

Different symbols or keywords create different variable types.

Check the type with:

```
class(x)
```

Examples students can type in MATLAB

1) Number → double

```
x = 10;  
class(x)
```

```
ans = double
```

2) Single quotes → char

```
a = 'A';  
class(a)
```

```
ans = char
```

3) Double quotes → string

```
b = "Hi";  
class(b)
```

```
ans = string
```

4) true/false → logical

```
p = true;  
class(p)
```

```
ans = logical
```

What creates each type?

Use this	MATLAB type
normal number	double
single quotes ' '	char
double quotes " "	string
true / false	logical
int8(25)	int8

Quick note

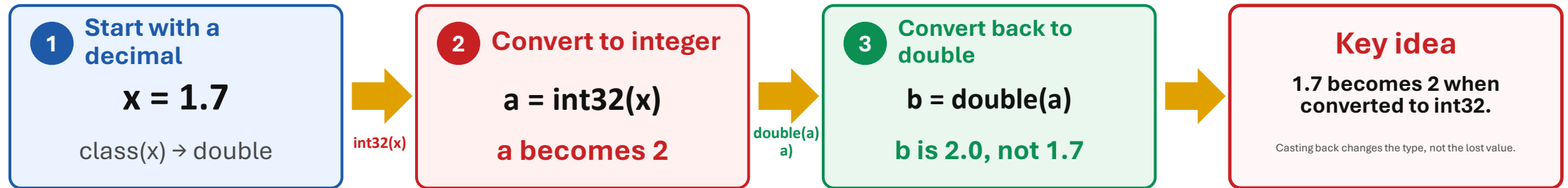
Normal numbers are double by default. Use special functions like `int8()` only when you really need an integer type.

TAKEAWAY: Use `class()` whenever you are not sure what type MATLAB created.

Type Casting in MATLAB

Changing the data type can also change the value.

Type casting = converting a value from one data type to another, for example from double to int32.



Live MATLAB demo

```
x = 1.7;  
a = int32(x); % rounds to 2  
b = double(a); % type becomes double again  
whos x a b
```

Expected result

Name	Value	Class
x	1.7000	double
a	2	int32
b	2.0000	double

Why is b not 1.7?

Because the decimal part was lost when x was cast to int32.

Main message: Type casting can change both the class and the value. Be careful when converting decimals to integers.

Type Casting: Numbers and Logicals

There are many functions that convert values from one type to another. The names of these functions are the same as the names of the types. The class names can be used as functions to convert a value to that type. This is called type casting.

```
x = 0;  
a = logical(x)  
  
y = -7;  
b = logical(y)  
  
c = double(b)
```

: Numeric and logical values are easily interchangeable, for good or ill. Adding a logical value to a numeric value returns a double. Type casting any numeric value to a logical will return 0 if the double is equal to 0 and 1 if it is not.

Type Casting: Numbers and Strings

Due to constraints in receiving input data, it is often useful to transform between characters, strings, and numbers. Strings can be transformed to numeric values using class name conversions. Characters must be converted to numbers (vice versa) using the functions `num2str(.)` and `str2num(.)`.

ASCII Character Conversion

```
x = double('9')
y = char(x)

x =

    57

y =

    '9'
```

Face Value Conversion

```
x = str2num('9')
y = num2str(x)

x =

    9

y =

    '9'
```

: Character variables look at each letter individually and map it to a specific ASCII code. Type casting with a class name conversion from char -> num will return the ASCII code for each character. Type casting with a class name conversion from num -> char will return the ASCII character associated with that number.

```
x = '9'
class(x)

a = double(x)
b = str2num(x)
c = num2str(b)
class(c)
```

The ASCII Table

code (pointing to Dec) *picture* (pointing to Char)

Dec	Char	Dec	Char	Dec	Char	Dec	Char	Dec	Char
32	space	52	4	72	H	92	\	112	p
33	!	53	5	73	I	93	]	113	q
34	"	54	6	74	J	94	^	114	r
35	#	55	7	75	K	95	_	115	s
36	\$	56	8	76	L	96	`	116	t
37	%	57	9	77	M	97	a	117	u
38	&	58	:	78	N	98	b	118	v
39	'	59	;	79	O	99	c	119	w
40	(	60	<	80	P	100	d	120	x
41	)	61	=	81	Q	101	e	121	y
42	*	62	>	82	R	102	f	122	z
43	+	63	?	83	S	103	g	123	{
44	,	64	@	84	T	104	h	124	
45	-	65	A	85	U	105	<u>i</u>	125	}
46	.	66	B	86	V	106	j	126	~
47	/	67	C	87	W	107	k	127	DEL
48	0	68	D	88	X	108	l		
49	1	69	E	89	Y	109	m		
50	2	70	F	90	Z	110	n		
51	3	71	G	91	[	111	o		

double('3') (pointing to 51) *'3' + 27 = 78* (pointing to 78)

Working with Variables

- Operators are used to perform actions on variables which change their value, type, and size. You can think of operators as your ***basic calculator buttons***.
- There are in general four kinds of operators:
 - *Construction operators, which are used to generate variables of specific types and sizes (name = 'Ali'; numbers = [1 2 3])*
 - *unary operators, which modify on a single value (x = -5)*
 - *binary operators, which combine two values in a designated way (x = 5 + 3; y = 10 * 2)*
 - *Specialized operators, which have distinct roles in grouping operations (x = (5 + 3) * 2).*

Numeric Operators

Operators that Work with Numbers to Generate Numbers

- Unary Operators Include:
 - negation
- Binary Operators include:
 - + addition
 - subtraction
 - * multiplication
 - / division
 - ^ exponentiation
- Specialized Operators Include:
 - () grouping

```
val1 = 5 + 1;  
val2 = 10*2;  
val3 = 10^2;  
val4 = 100/5;  
val5 = 10 + 2 + 3;
```

Name	Value
val1	6
val2	20
val3	100
val4	20
val5	15

Operator Precedence Rules

Highest



Lowest

()	parentheses
^	exponentiation
-	negation
* / \	all multiplication and division
+ -	addition and subtraction

- Within given precedence, the expressions are evaluated from **left to right**.
- Nested parentheses: expressions in inner parentheses are evaluated first

```
val1 = (5 + 1) * 2;  
val2 = 10 ^ 2 * 3;  
val3 = 10 ^ (2 + 3);  
val4 = 4 - 3 * 2;  
val5 = 3 * ((4 + 3) * 2);  
val6 = -10 ^ 2;
```

Name	Value
val1	12
val2	300
val3	100,000
val4	-2
val5	42
val6	-100

Relational Operators

Operators that Work with Numbers to Generate Logicals

Relational operators allow us to ask questions about our numeric variables, to which MatLab will provide the answer of **true** (logical 1) or **false** (logical 0).

- Binary Operators include:
 - > greater than
 - < less than
 - >= greater than or equals
 - <= less than or equals
 - == equality
 - ~= inequality
- Specialized Operators Include:
 - () grouping

```
ro1 = 3 < 4;  
ro2 = 3 > 3;  
ro3 = 3 == 5;  
ro4 = 3 ~= 7;
```

Name	Value
ro1	1
ro2	0
ro3	0
ro4	1

More Operator Precedence Rules

Highest



Lowest

()	parentheses
^	exponentiation
-	negation
* / \	all multiplication and division
+ -	addition and subtraction
<, > <=, >= ==, ~=	all relational operators

val1 = (3 < 4) < 4;
val2 = 3 < (4 < 5);
val3 = 3 < (1 + 3);
val4 = (3 < 1) + 3;
val5 = 3 < 1 + 3;

⚠: Although there is no error for computing val5, this syntax is not recommended to avoid a potential mistake.

Name	Value	Type
val1	1	logical
val2	0	logical
val3	1	logical
val4	3	double
val5	1	logical

Logical Operators

Operators that Work with Logicals to Generate Logicals

Logical operators allow us to **combine the questions we** ask about our numeric variables, yielding a result of **true** (logical 1) or **false** (logical 0) for the combined condition.

- Unary Operators include:

~ not

- Binary Operators include:

|| or
&& and

- Specialized Operators Include:

() grouping

- Note that the logical operators are **commutative**

▪ (e.g., $x || y$ is equivalent to $y || x$)

is either variable true
are both variables true

x	y	~x	~y	x y	x && y
true	true	0	0	1	1
false	true	1	0	1	0
true	false	0	1	1	0
false	false	1	1	0	0

"and" ≠ inverse of "or"

Given

More Operator Precedence Rules

Highest



Lowest

()	parentheses
^	exponentiation
- ~	unary operators
* / \	all multiplication and division
+ -	addition and subtraction
<, > <=, => ==, ~=	all relational operators
&&	and
	or

```
lg1 = (10 > 4) && (4 > 1);  
lg2 = (10 < 4) && (4 < 1);  
lg3 = ~((10 < 4) && (4 < 1));  
lg4 = 3 < 10 < 7;  
lg5 = 3 < 10 && 10 < 7;
```

Name	Value
lg1	1
lg2	0
lg3	1
lg4	1
lg5	0

String Operators

Operators that Work with Strings to Generate Strings

- Binary Operators include:
 - + addition
- Specialized Operators Include:
 - () grouping

: While string variables can be added together, character variables do not work with any operators directly. The use of numerical and relational operators on a character variable will enact an *implicit type cast* of the character variable to a numerical variable.

```
str1 = "Hello";  
str2 = "World";  
str3 = str1 + str2;  
str4 = str1 + " " + str2;  
str5 = str4 + "!";
```

Name	Value
str1	Hello
str2	World
str3	HelloWorld
str4	Hello World
str5	Hello World!

Announcements

- Academic Integrity Quiz -- Due tomorrow's night at 11 pm
 - You must get all questions correct to receive credit.
- Homework #1 will be posted on LEARN after class
 - Complete all problems on your own instance of MatLab.
 - Submit problems 1-3 on MatLab Grader.
 - Submit problem 4 to CrowdMark.
 - Submit all code files to the LEARN dropbox.